noisy measurement, especially with modern multicore architectures: it's going to be hard to sandbox your experiments, and both the operating system and background processes will influence the result.

But this is not even the worst part: this result will be largely influenced by the hardware you are running your experiments on, so it won't have any meaning as an absolute number. You might try to run a benchmark test, and compare your algorithm performance to some other known algorithm. Sometimes this technique produces meaningful results, but... doesn't seem the way to go, because there would still be too many variables to take into consideration, from the hardware and operating system it’s running on to the type and size of input used in your tests.

Continuing in your reasoning, you can then think about counting the instruction ran, instead. This is probably a good indicator of how much time it will be needed to run the whole algorithm, and sufficiently generalizable, right?

Well... not quite.

- Will you count machine instructions? If so, it will not be platform-agnostic.
- If, instead, you count high-level instructions only, it will heavily depend on the programming language you choose: *Scala* or *Nim* can be much more succinct than *Java*, *Pascal*, or *COBOL*.
- What can be considered an improvement? Is that relevant if your algorithm runs 99 instructions instead of 101?